



**RAMAIAH**  
Institute of Technology

# **Department of Information Science & Engineering**

**Term: 10th Sep 2025 – 31st Dec 2025**

**Semester: 5**

**Subject Code: IS53**

**Credits: 4:0:0**

**Subject Name: Computer Networks**

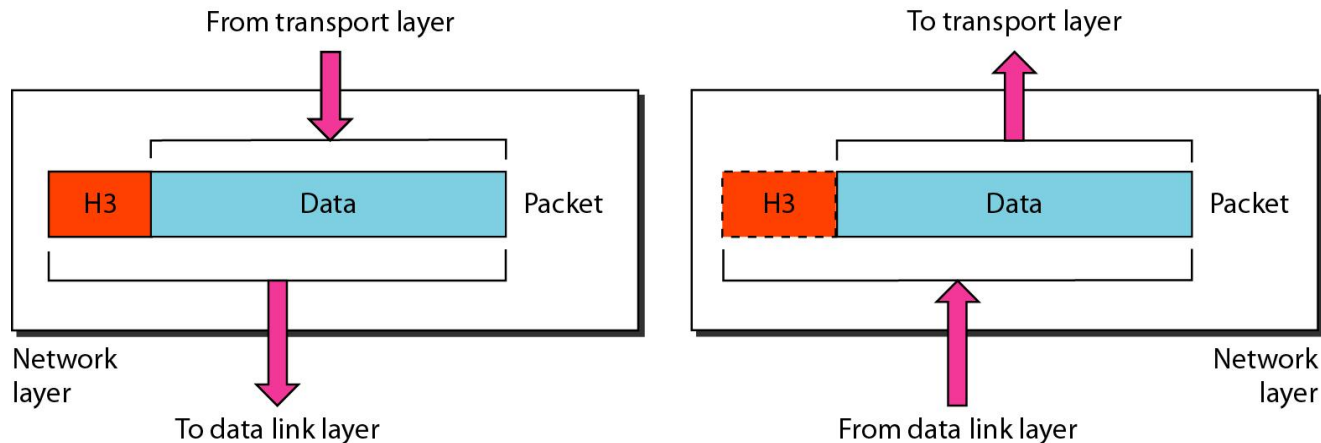
# **UNIT 3**

## **NETWORK LAYER**

# NETWORK LAYER

- ✓ Data link layer operates on the same network(link)
- ✓ Network layer operates on different networks(links)
- ✓ Responsible for source to destination delivery

**The network layer is responsible for the delivery of individual packets from the source host to the destination host.**



---

# **Responsibilities / Functions / Services of Network Layer**

---

- **Source to destination delivery**
  - **Logical addressing**
  - **Packetizing**
  - **Routing and Forwarding**
  - **Other services**
    - **Error Control , Flow control**
    - **Congestion control, Quality of service**
    - **Security**
-

---

# PACKET SWITCHING

---

- **Datagram Approach – Connection less service**
  - **Virtual Circuit Approach – Connection Oriented service**
    - **Setup Phase (Request packet & ACK packet)**
    - **Data Transfer Phase**
    - **Tear down / Termination phase**
-

# NETWORK-LAYER PERFORMANCE

---

## ➤ Delay

- Transmission delay

- Propagation delay

- Processing delay

- Queuing delay

- Total delay – Sum of the above

## ➤ Throughput

- Packet loss

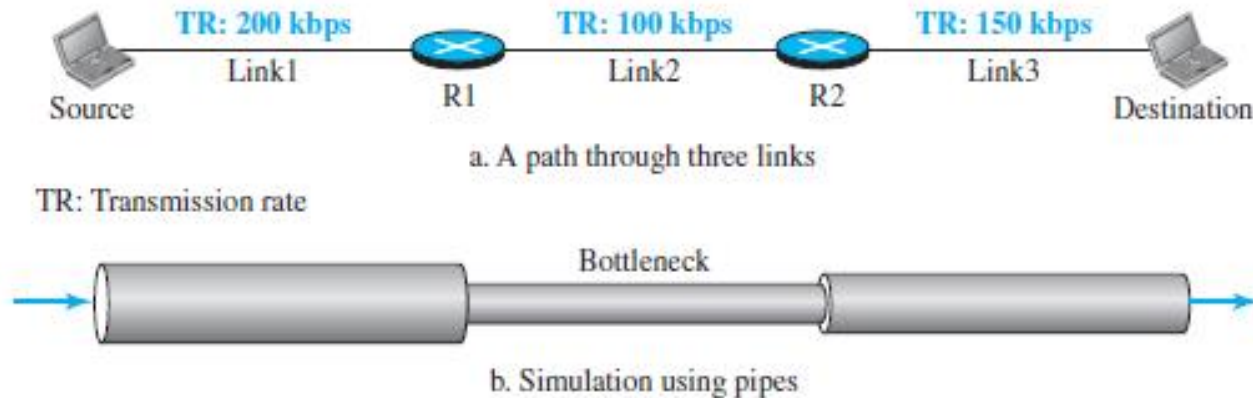
- Congestion

# DELAY

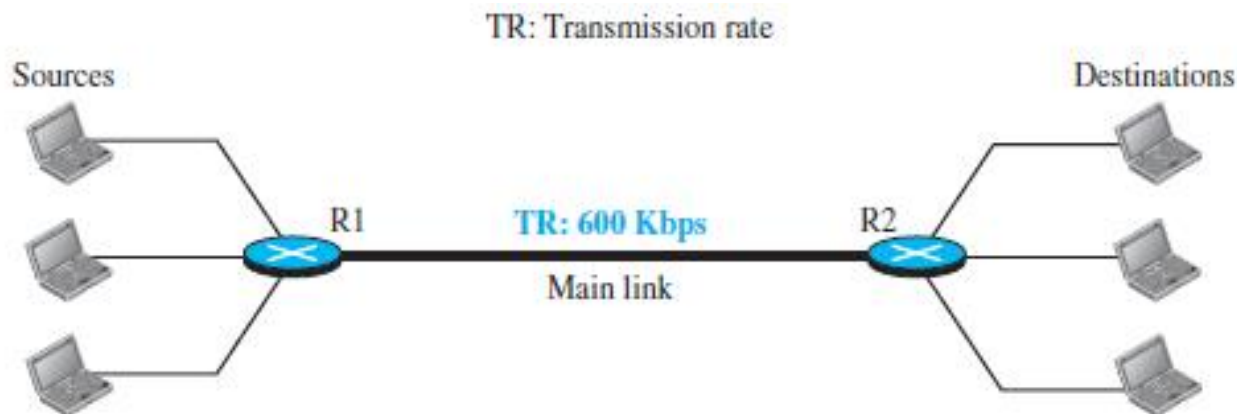
---

- **Transmission delay:**  $\text{Delay}_{\text{tr}} = (\text{Packet length}) / (\text{Transmission rate})$
  - **Propagation delay:**  $\text{Delay}_{\text{pg}} = (\text{Distance}) / (\text{Propagation speed})$
  - **Processing delay:**  $\text{Delay}_{\text{pr}} = \text{Time required to process a packet in a router or a destination host}$
  - **Queuing delay:**  $\text{Delay}_{\text{qu}} = \text{The time a packet waits in input and output queues in a router}$
  - **Total delay:** Sum of the above
- $\text{Total delay} = (n+1) (\text{Delay}_{\text{tr}} + \text{Delay}_{\text{pg}} + \text{Delay}_{\text{pr}}) + (n) (\text{Delay}_{\text{qu}})$

# THROUGHPUT



➤  $\text{Throughput} = \text{minimum } \{TR_1, TR_2, \dots, TR_n\}$



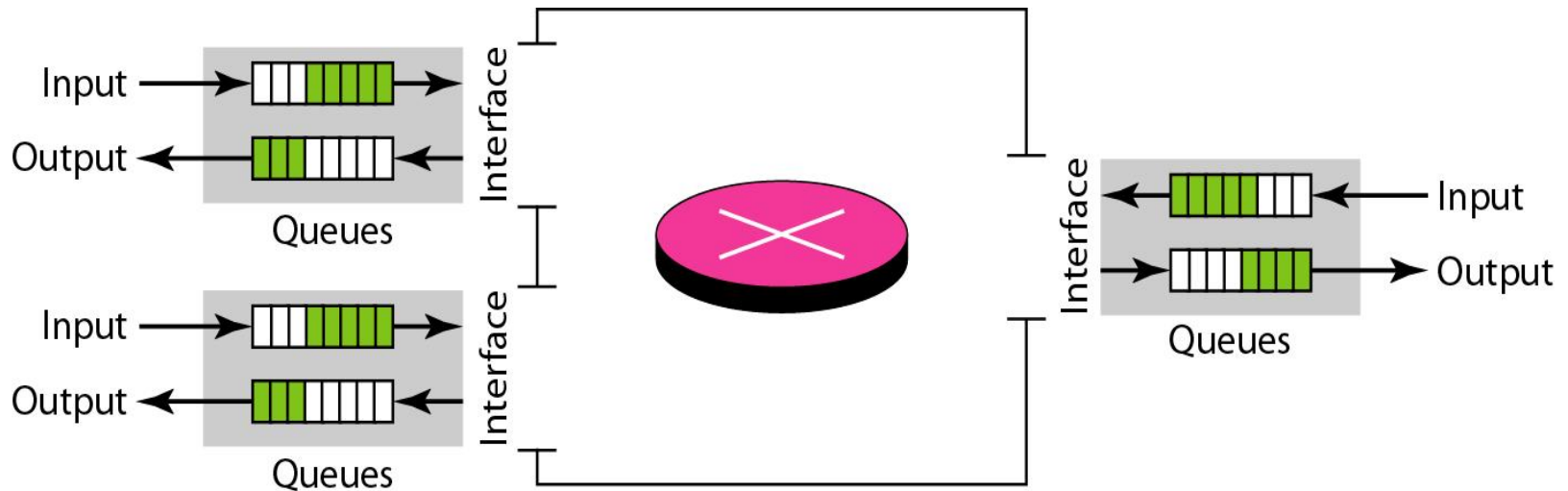
**Effect of throughput in shared links**

# Congestion Control

---

- **Congestion:**
  - the load on the network is greater than the capacity of the network
- **Congestion control:**
  - the mechanisms to control the congestion and keep the load below the capacity
- Congestion happens in any system that involves waiting
- Congestion occurs because routers and switches have queues/buffers that hold the packets before and after processing

# Congestion Control



## Two Issues

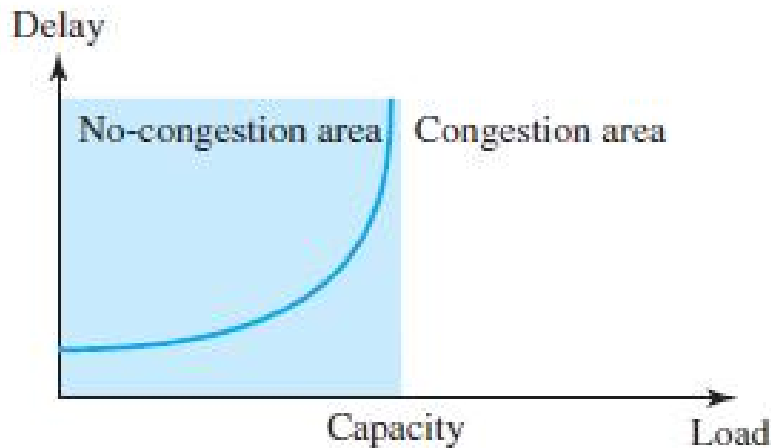
- The rate of packet arrival  $>$  packet processing rate  $\rightarrow$  affects input queue
- The packet departure rate  $<$  packet processing rate  $\rightarrow$  affects output queue

- Congestion control involves two factors that measure the performance of a network - Delay and throughput
- Delay is composed of propagation delay and processing delay
- Throughput: the number of packets passing through the network in a unit of time

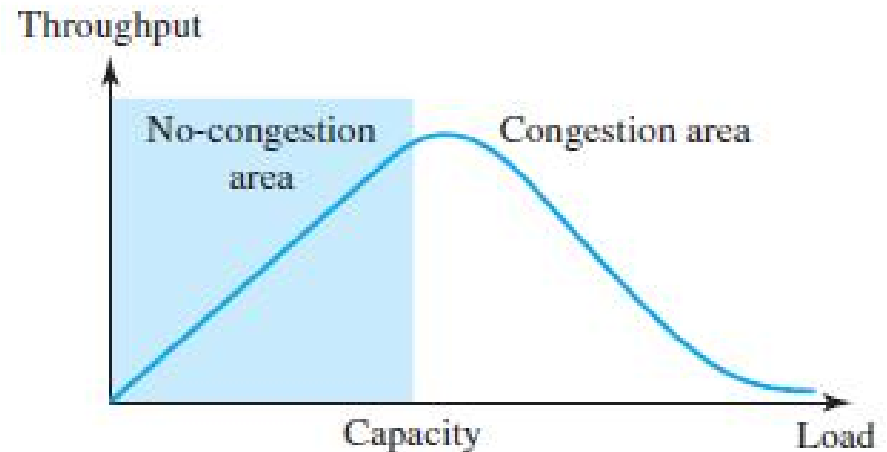
---

*Packet delay and throughput as functions of load*

---



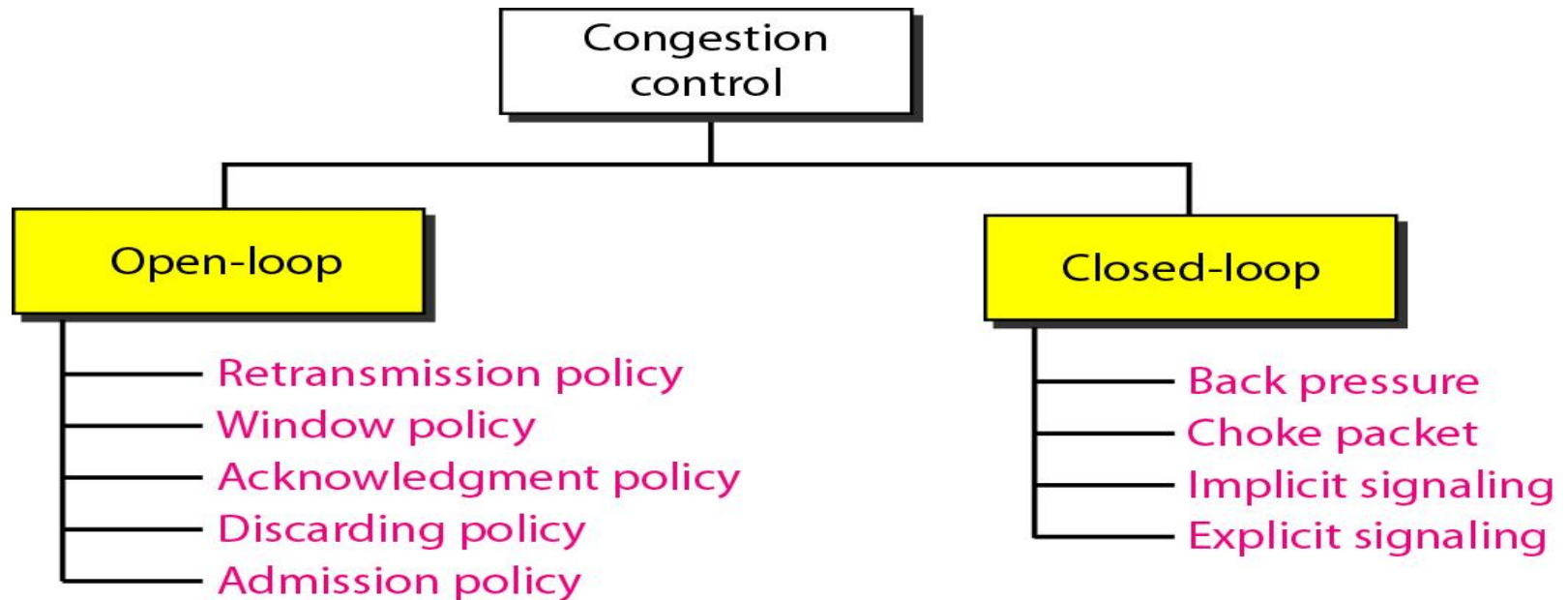
a. Delay as a function of load



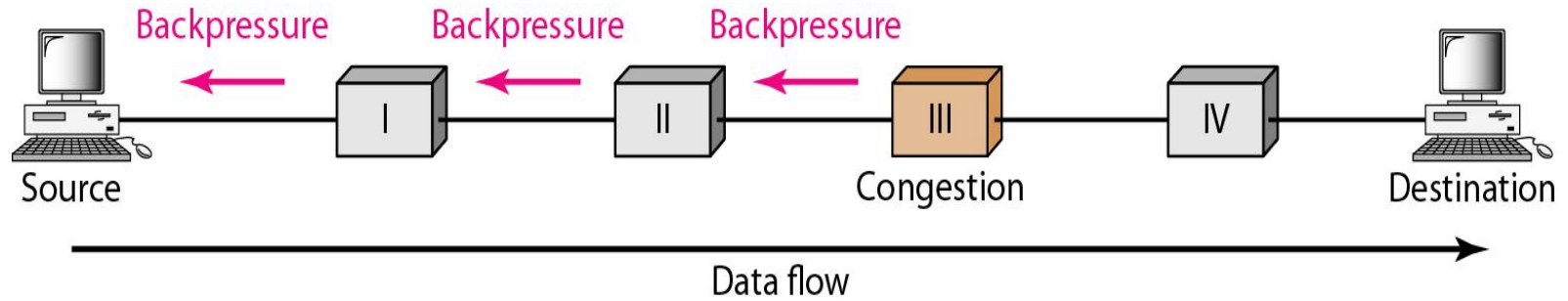
b. Throughput as a function of load

# Congestion Control

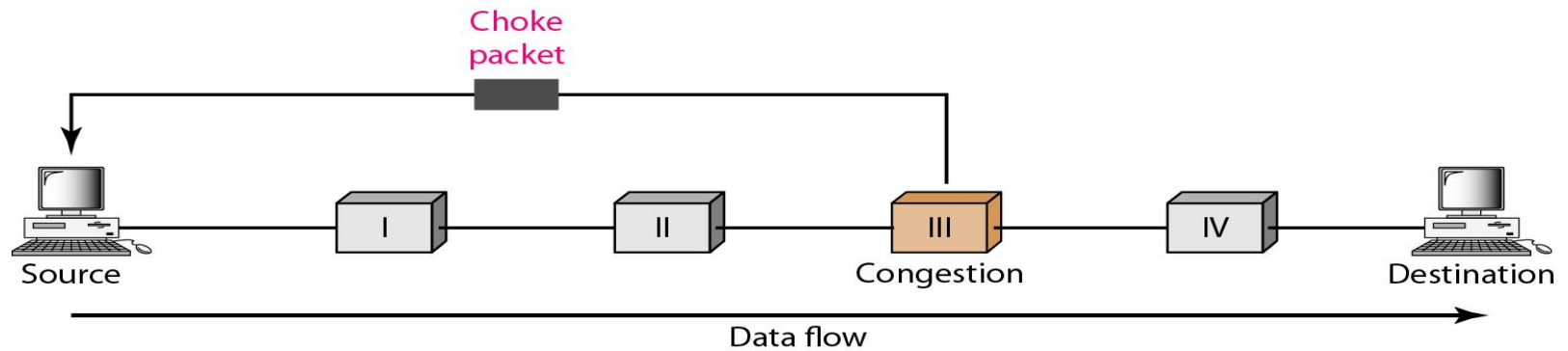
- Congestion control refers to techniques and mechanisms that can either prevent congestion or remove congestion.
- Two broad categories: **open-loop congestion control** (prevention) and **closed-loop congestion control** (removal).



# Back pressure



# Choke Packet



# IPv4 ADDRESSES



# IPv4 ADDRESSES

*An **IPv4 address** is a **32-bit** address that uniquely and universally defines the connection of a device (for example, a computer or a router) to the Internet.*

**An IPv4 address is 32 bits long**

**The IPv4 addresses are unique and universal**

**The address space of IPv4 is  $2^{32}$  or  
4,294,967,296 (4 billion+)**

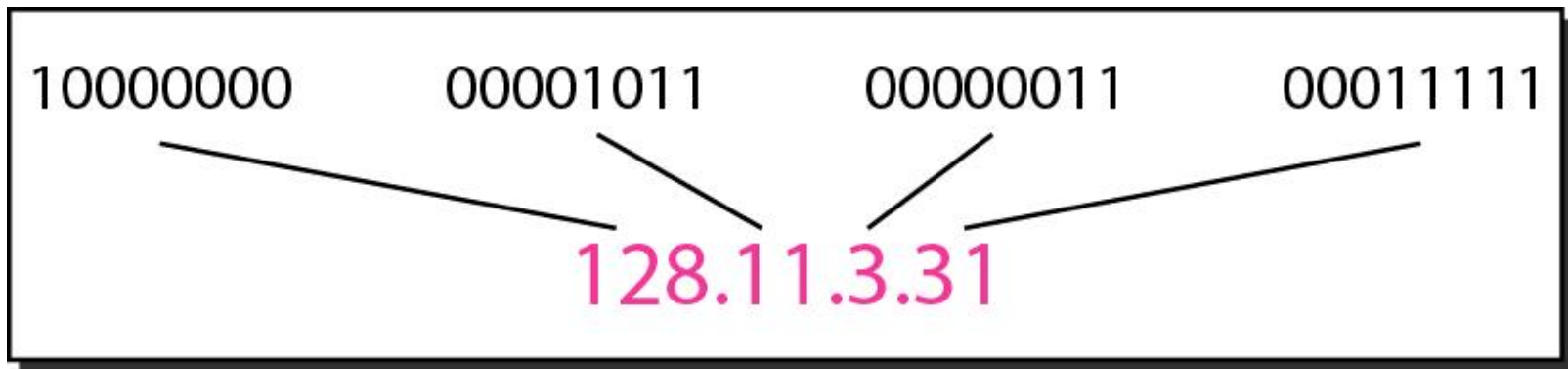
# IPv4 ADDRESS NOTATION

## *Binary Notation :*

- ✓ Address is displayed as 32 bits.
- ✓ Referred to as 32-bit address or 4-byte address
- ✓ Example: 01110101 10010101 00011101 00000010

## *Dotted Decimal Notation :*

- ✓ More compact and easier to read
- ✓ Example: 117.149.29.2



# IPv4 ADDRESS NOTATION

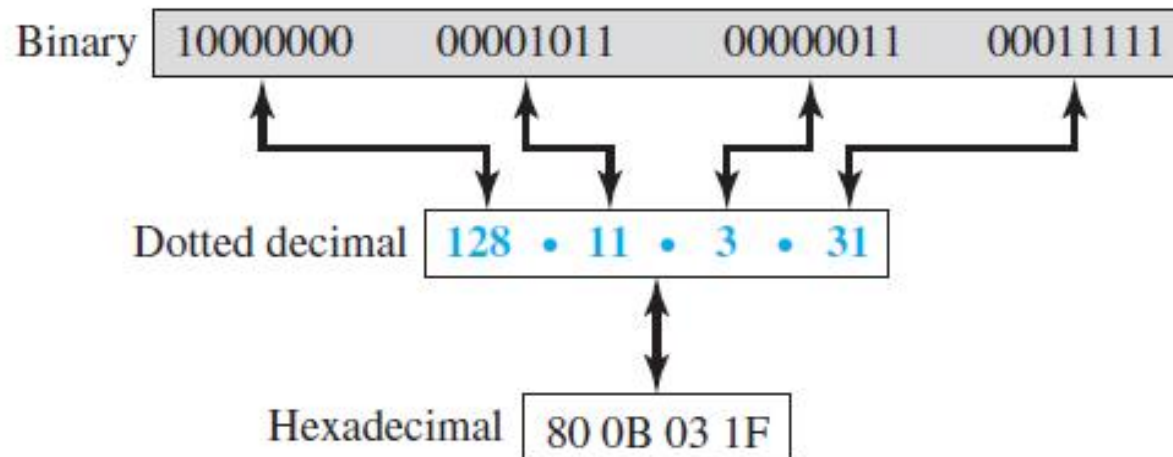
## Hexadecimal Notation :

- ✓ 8 hexadecimal digits
- ✓ Example: 80 0B 03 1F

---

*Three different notations in IPv4 addressing*

---





## *Example 1*

*Change the following IPv4 addresses from binary notation to dotted-decimal notation.*

a. 10000001 00001011 00001011 11101111

b. 11000001 10000011 00011011 11111111

### *Solution*

*We replace each group of 8 bits with its equivalent decimal number and add dots for separation.*

a. 129.11.11.239

b. 193.131.27.255



## *Example 2*

*Change the following IPv4 addresses from dotted-decimal notation to binary notation.*

a. 111.56.45.78

b. 221.34.7.82

### *Solution*

*We replace each decimal number with its binary equivalent*

a. 01101111 00111000 00101101 01001110

b. 11011101 00100010 00000111 01010010



### ***Example 3***

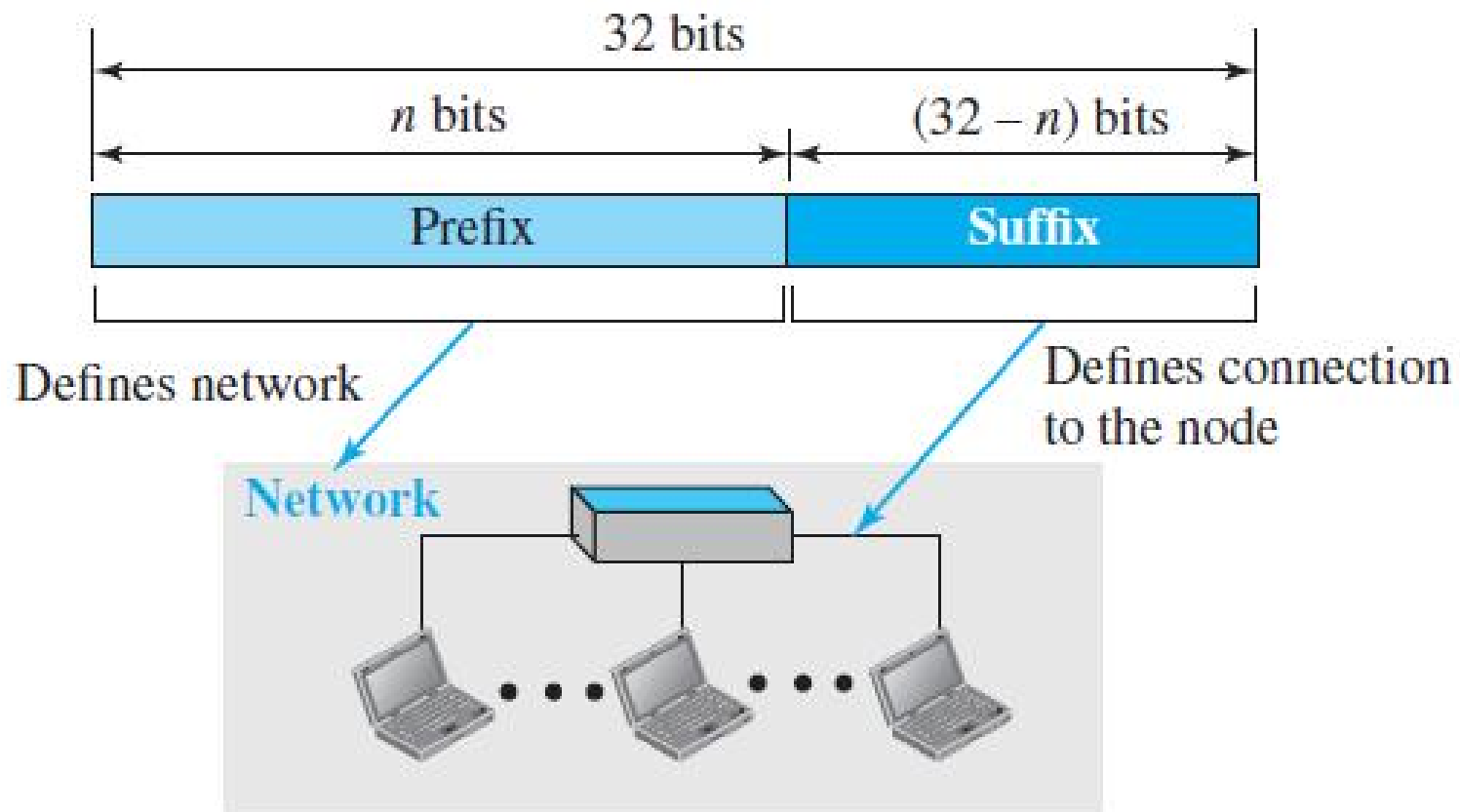
***Find the error, if any, in the following IPv4 addresses.***

- a.*** 111.56.045.78
- b.*** 221.34.7.8.20
- c.*** 75.45.301.14
- d.*** 11100010.23.14.67

### ***Solution***

- a.*** ***There must be no leading zero (045).***
- b.*** ***There can be no more than four numbers.***
- c.*** ***Each number needs to be less than or equal to 255.***
- d.*** ***A mixture of binary notation and dotted-decimal notation is not allowed.***

# HIERARCHY IN ADDRESSING



# IPv4 CLASSFUL ADDRESSING

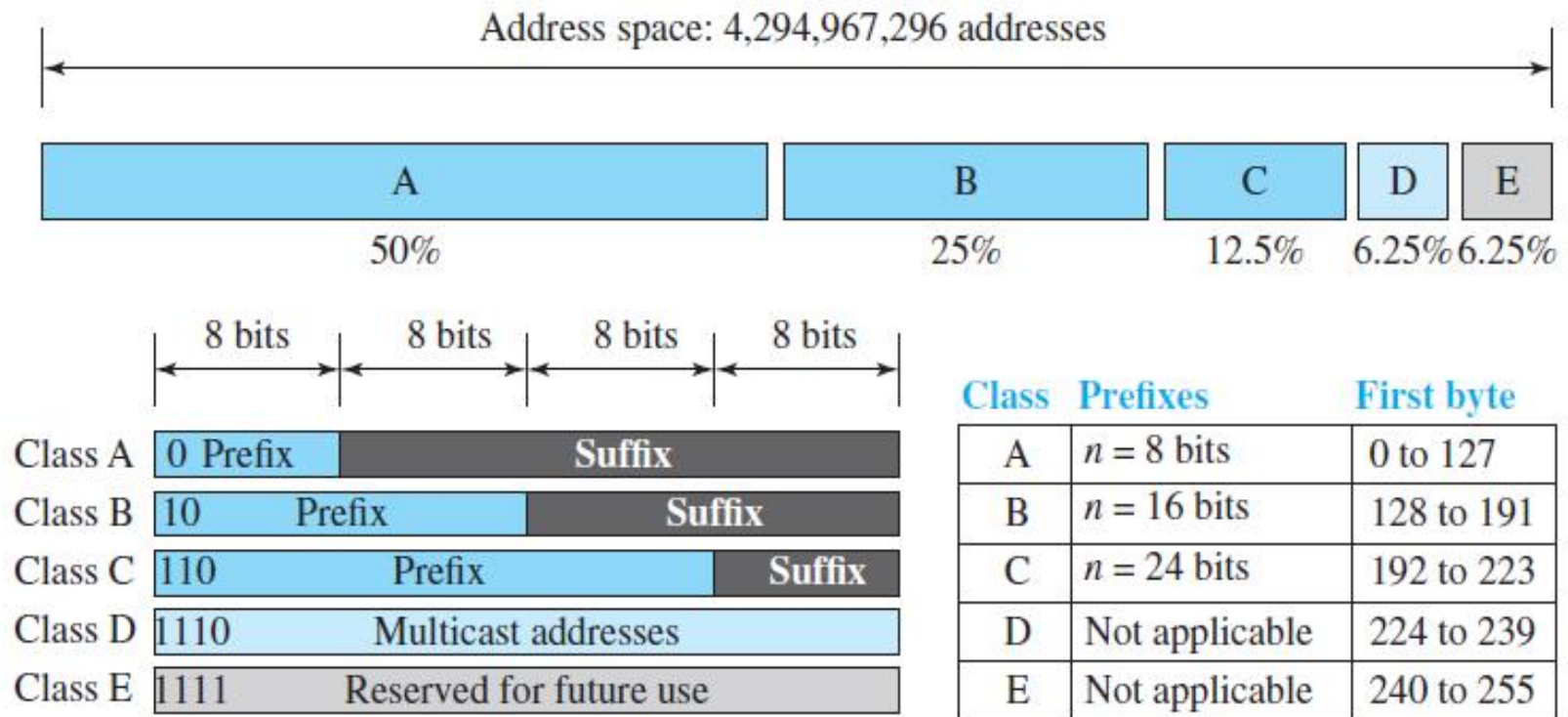
In classful addressing, the address space is divided into five classes: A, B, C, D, and E.

Each IP address consists of 4 bytes, defining three fields :

- ✓ Class Type
- ✓ NetID (network identifier)
- ✓ HostID (host identifier)



## Occupation of the address space in classful addressing



| Class | Number of Addresses      | Percentage |
|-------|--------------------------|------------|
| A     | $2^{31} = 2,147,483,648$ | 50%        |
| B     | $2^{30} = 1,073,741,824$ | 25%        |
| C     | $2^{29} = 536,870,912$   | 12.5%      |
| D     | $2^{28} = 268,435,456$   | 6.25%      |
| E     | $2^{28} = 268,435,456$   | 6.25%      |

# ***Finding the classes in binary and dotted-decimal notation***

|         | First<br>byte | Second<br>byte | Third<br>byte | Fourth<br>byte |
|---------|---------------|----------------|---------------|----------------|
| Class A | 0             |                |               |                |
| Class B | 10            |                |               |                |
| Class C | 110           |                |               |                |
| Class D | 1110          |                |               |                |
| Class E | 1111          |                |               |                |

a. Binary notation

|         | First<br>byte | Second<br>byte | Third<br>byte | Fourth<br>byte |
|---------|---------------|----------------|---------------|----------------|
| Class A | 0-127         |                |               |                |
| Class B | 128-191       |                |               |                |
| Class C | 192-223       |                |               |                |
| Class D | 224-239       |                |               |                |
| Class E | 240-255       |                |               |                |

b. Dotted-decimal notation



## *Example 4*

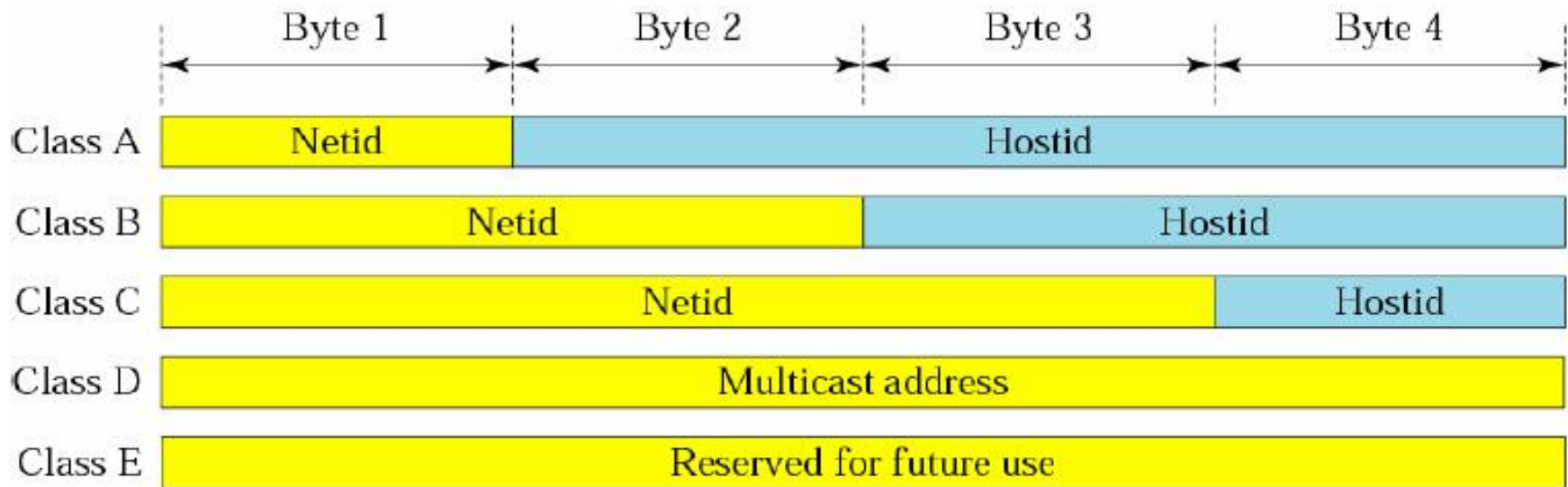
*Find the class of each address.*

- a.* 00000001 00001011 00001011 11101111
- b.* 11000001 10000011 00011011 11111111
- c.* 14.23.120.8
- d.* 252.5.15.111

*Solution*

- a.* The first bit is 0. This is a class A address.
- b.* The first 2 bits are 1; the third bit is 0. This is a class C address.
- c.* The first byte is 14; the class is A.
- d.* The first byte is 252; the class is E.

# Netid and hostid



- Only classes A, B, and C addresses are subdivided.
- Exercise: How many different “Netid”s and “Host”s in each of the classes A, B, C?

## Netid's

- $2^{(8-1)} = 128$
- $2^{(16-2)} = 16,384$
- $2^{(24-3)} = 2,097,152$

## Hostid's

- $2^{24} = 16,777,216$
- $2^{16} = 65,536$
- $2^8 = 256$

## *Number of blocks and block size in classful IPv4 addressing*

| <i>Class</i> | <i>Number of Blocks</i> | <i>Block Size</i> | <i>Application</i> |
|--------------|-------------------------|-------------------|--------------------|
| A            | 128                     | 16,777,216        | Unicast            |
| B            | 16,384                  | 65,536            | Unicast            |
| C            | 2,097,152               | 256               | Unicast            |
| D            | 1                       | 268,435,456       | Multicast          |
| E            | 1                       | 268,435,456       | Reserved           |

**Class A address:** designed for large organizations with a large number of attached hosts or routers. (wasted & not used)

**Class B address:** designed for midsize organizations with ten of thousands of attached hosts or routers( too large for many organizations)

**Class C address:** designed for small organizations with a small number of attached hosts or routers.(too small for many organizations)

**Class D address:** designed for multicasting (waste of addresses)

**Class E address:** reserved for future use (waste of addresses)



**Subnetting**

**Address Depletion**

**Supernetting**



*Note*

**Classful addressing, which is almost obsolete, is replaced with classless addressing.**

# CLASSLESS ADDRESSING

- ✓ Serves as the solution for address depletion but does not remove it
- ✓ No classes but the address are still granted in blocks
- ✓ The size of the block( the number of addresses) varies based on the nature and size of the entity.
  - Household: 2 addresses
  - Large organization: thousands of addresses.
  - ISP: thousands or hundreds of thousands based on the number of customers it may serve.

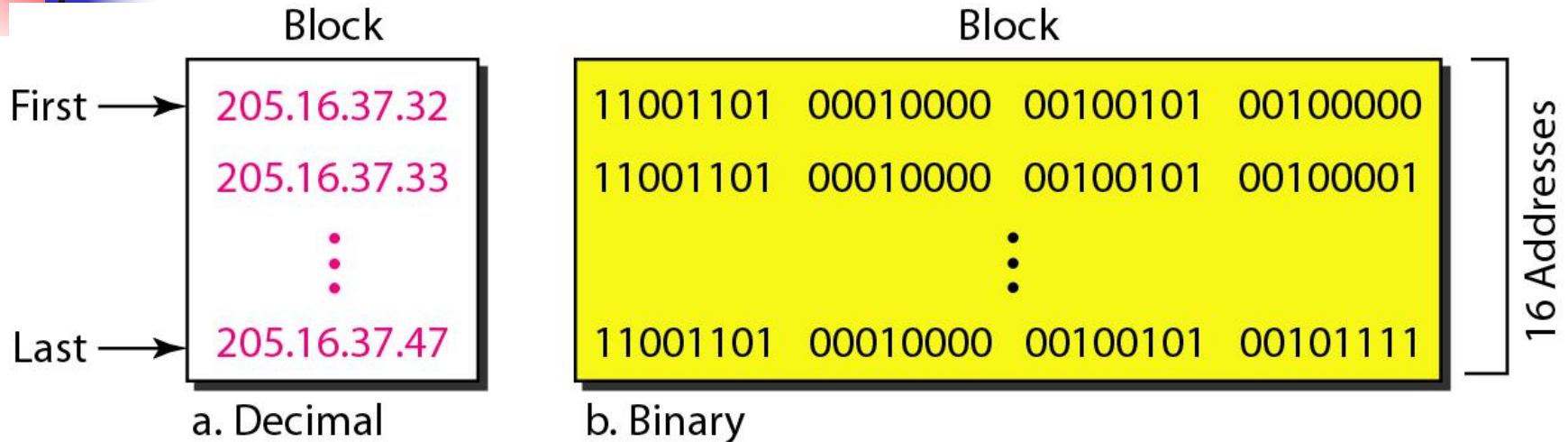
# CLASSLESS ADDRESSING

## Restriction

Internet authorities impose three restrictions on classless address blocks:

1. The addresses in a block must be contiguous, one after another.
2. The number of addresses in a block must be a power of 2 (1, 2, 4, 8, ... ).
3. The first address must be evenly divisible by the number of addresses.

# *Classless Address Block – Example*



The restrictions are applied to this block:

- ✓ The addresses are contiguous.
- ✓ The number of addresses is a power of 2 ( $2^4=16$ )
- ✓ The first address is divisible by 16. The first address, when converted to a decimal number is 3,440,387,360 which when divided by 16 results in 215,024,210.

# MASK - CLASSLESS ADDRESSING

- ✓ Mask: 32- bit
  - The n leftmost bits are 1s
  - 32-n rightmost bits are 0s
- ✓ Prefix length – slash notation
- ✓ In CIDR, n has a value from 0 to 32 and used to show the mask in the form /n.
- ✓ One of the address and the /n notation define the whole address block:
  - First address
  - Last address
  - Number of address



***Note***

**In IPv4 classless addressing, a block of addresses can be defined as**

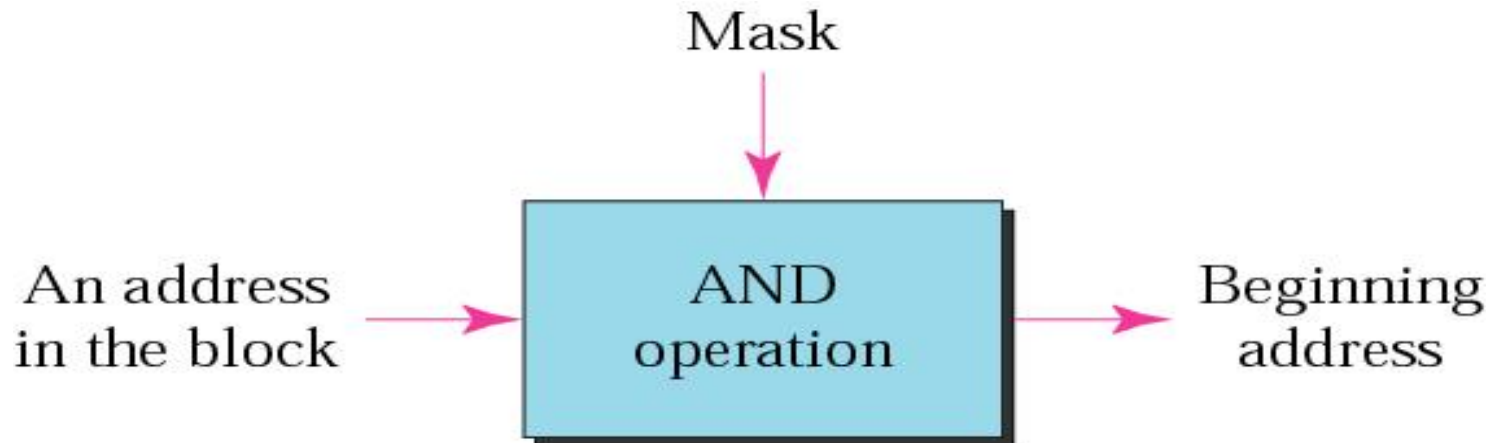
**$x.y.z.t / n$**

**in which  $x.y.z.t$  defines one of the addresses and the  $/n$  defines the mask.**

**Example: 172.31.1.2 / 25**

## Note

**The first address in the block can be found by setting the rightmost  $32 - n$  bits to 0s.**



**The first address in the block can be found by AND - ing the given address with the mask.**



## *Example 6*

*A block of addresses is granted to a small organization. We know that one of the addresses is 205.16.37.39/28. What is the first address in the block?*

### *Solution*

*The binary representation of the given address is*

*11001101 00010000 00100101 00100111*

*If we set 32–28 rightmost bits to 0, we get*

*11001101 00010000 00100101 00100000*

*or*

*205.16.37.32/28*

*This is the one shown in the classless address block example.*



*Note*

**The last address in the block can be found by setting the rightmost  $32 - n$  bits to 1s.**

**The last address in the block can be found by OR - ing the given address with the complement of the mask.**

## *Example 7*

*A block of addresses is granted to a small organization. We know that one of the addresses is 205.16.37.39/28. What is the last address in the block?*

### *Solution*

*The binary representation of the given address is*

*11001101 00010000 00100101 00100111*

*If we set 32 – 28 rightmost bits to 1, we get*

*11001101 00010000 00100101 00101111*

*or*

*205.16.37.47/28*

*This is the one shown in the classless address block example.*



*Note*

**The number of addresses in the block  
can be found by using the formula**

$$2^{32-n}.$$



## *Example*

---

*A block of addresses is granted to a small organization. We know that one of the addresses is 205.16.37.39/28. Find the number of addresses in the block ?*

## *Solution*

*The value of  $n$  is 28, which means that number of addresses is  $2^{32-28}$  or 16.*



## *Example – with mask ANDing , ORing*

*A block of addresses is granted to a small organization.  
We know that one of the addresses is 205.16.37.39/28.*

*Find*

- a. The first address*
- b. The last address*
- c. The number of addresses.*

## *Example (continued)*

### *Solution*

- a. The first address can be found by ANDing the given addresses with the mask. ANDing here is done bit by bit. The result of ANDing 2 bits is 1 if both bits are 1s; the result is 0 otherwise.*

|                |          |          |          |          |
|----------------|----------|----------|----------|----------|
| Address:       | 11001101 | 00010000 | 00100101 | 00100111 |
| Mask:          | 11111111 | 11111111 | 11111111 | 11110000 |
| First address: | 11001101 | 00010000 | 00100101 | 00100000 |

## *Example (continued)*

- b.** The last address can be found by ORing the given addresses with the complement of the mask. ORing here is done bit by bit. The result of ORing 2 bits is 0 if both bits are 0s; the result is 1 otherwise. The complement of a number is found by changing each 1 to 0 and each 0 to 1.*

|                  |          |          |          |          |
|------------------|----------|----------|----------|----------|
| Address:         | 11001101 | 00010000 | 00100101 | 00100111 |
| Mask complement: | 00000000 | 00000000 | 00000000 | 00001111 |
| Last address:    | 11001101 | 00010000 | 00100101 | 00101111 |



## *Example (continued)*

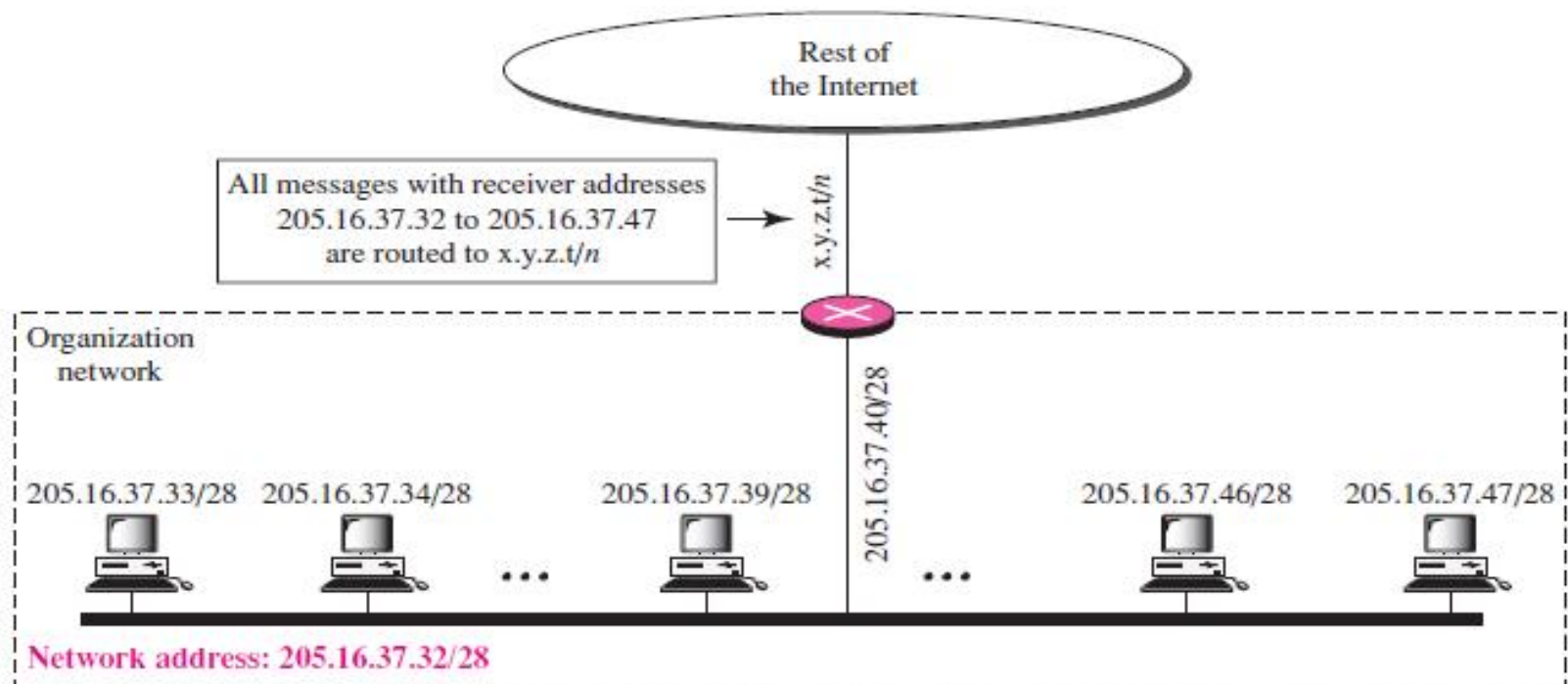
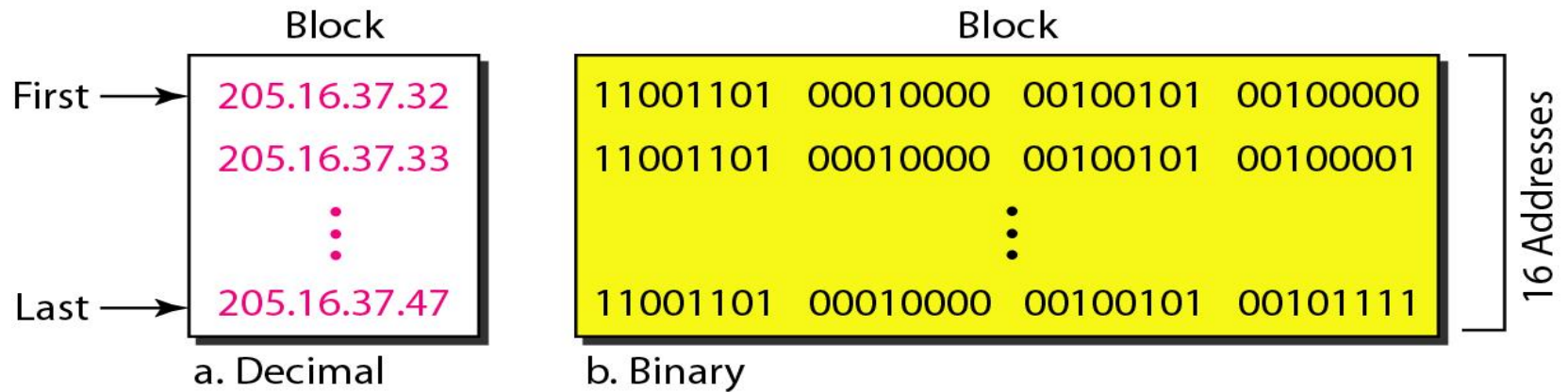
- c. The number of addresses can be found by complementing the mask, interpreting it as a decimal number, and adding 1 to it.*

Mask complement:      00000000 00000000 00000000 00001111

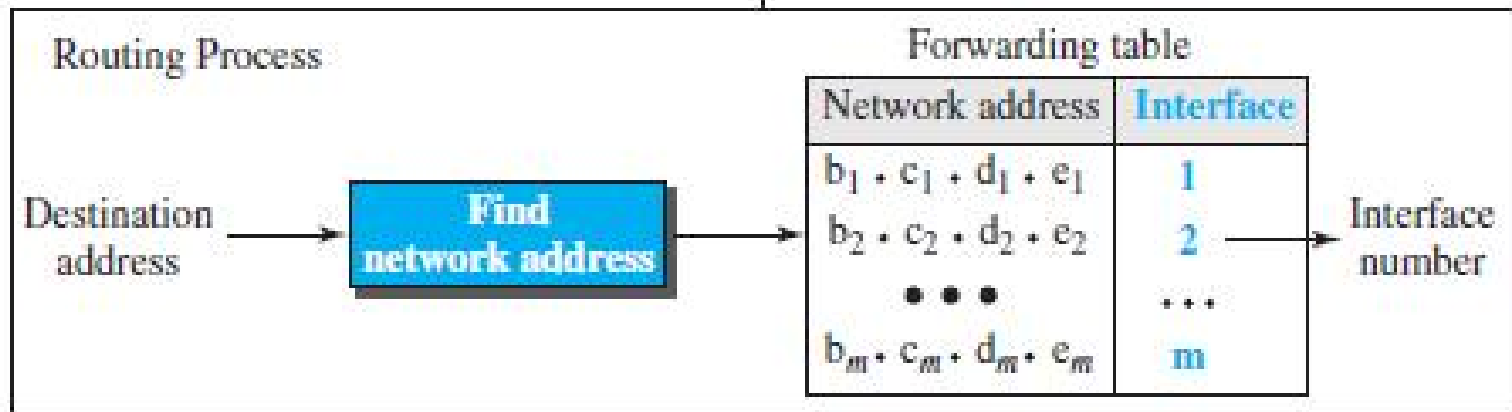
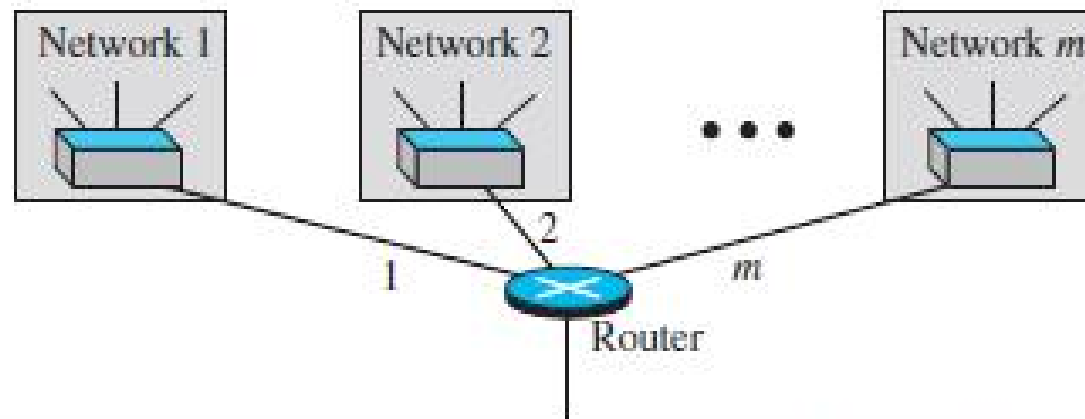
Number of addresses:     $15 + 1 = 16$

# NETWORK ADDRESS

- ✓ A very important concept in IP addressing is the network address.
- ✓ When an organization is given a block of addresses, the first address in the block is normally treated as a special address.
- ✓ The first address is called the network address and defines the organization network.
- ✓ Not assigned to any hosts or devices
- ✓ The network address represents the organization to the rest of the world.



## Network address



## *Example*

*Given the following classful address, find the network address?*

- a. 132.6.17.85*
- b. 23.56.7.91*

## *Solution*

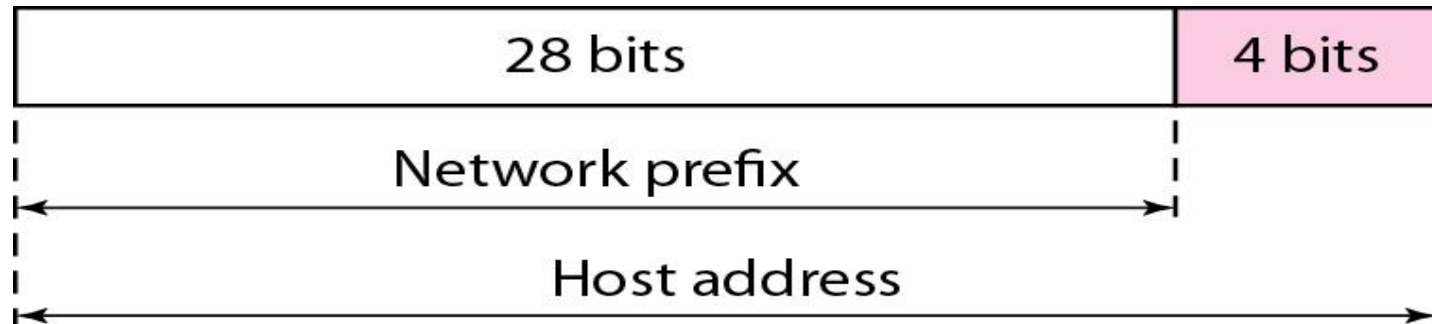
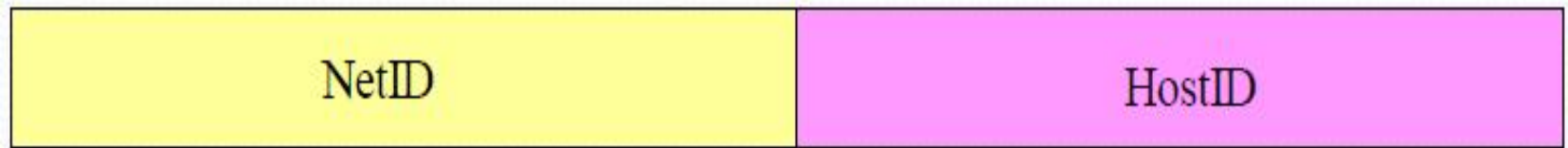
- a. The class is B. The first 2 bytes defines the netid. We can find the network address by replacing the hostid bytes (17.85) with 0s. **132.6.0.0.***
- b. The class is A. Only the first byte defines the netid. We can find the network address by replacing the hostid bytes (56.7.91) with 0s. **23.0.0.0.***

---

# *Hierarchy in Classless Addressing*

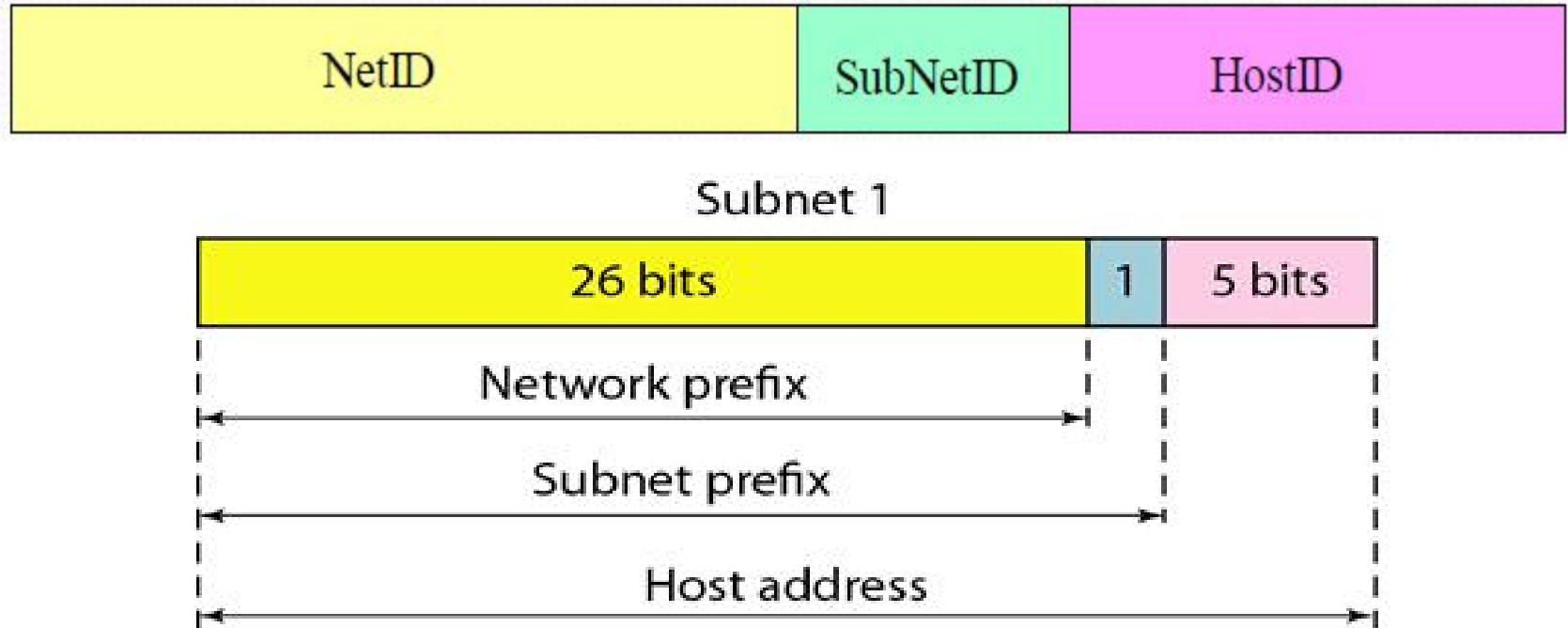
---

✓ Two Level Hierarchy – No Subnetting



# *Hierarchy in Classless Addressing*

✓ Three Level Hierarchy – Subnetting



✓ More Levels of Hierarchy – Subnets within subnet

# DESIGNING SUBNETS

- ✓ The subnetworks in a network should be carefully designed to enable the routing of packets.
- ✓ Lets assume -
  - ✓ Total number of addresses granted to the organization is  $N$ ,
  - ✓ Prefix length is  $n$ ,
  - ✓ Assigned number of addresses to each subnetwork is  $N_{\text{sub}}$ ,
  - ✓ Prefix length for each subnetwork is  $n_{\text{sub}}$ .

# DESIGNING SUBNETS

## Key requirements – Steps to follow

1. The addresses in a block must be contiguous, one after another.
2. The number of addresses in a block must be a power of 2 (1, 2, 4, 8, ... ).
3. Prefix length for each subnet is calculated using the formula:
$$n_{\text{sub}} = 32 - \log_2 N_{\text{sub}}$$
4. The first address must be evenly divisible by the number of addresses.

## ***Example 11***

*Suppose an organization is given the block 17.12.14.0/26, which contains 64 address. The organization has three offices and needs to divide the address into three subblocks of 32, 16 and 16 address. Design the configuration of the subnetted network.*

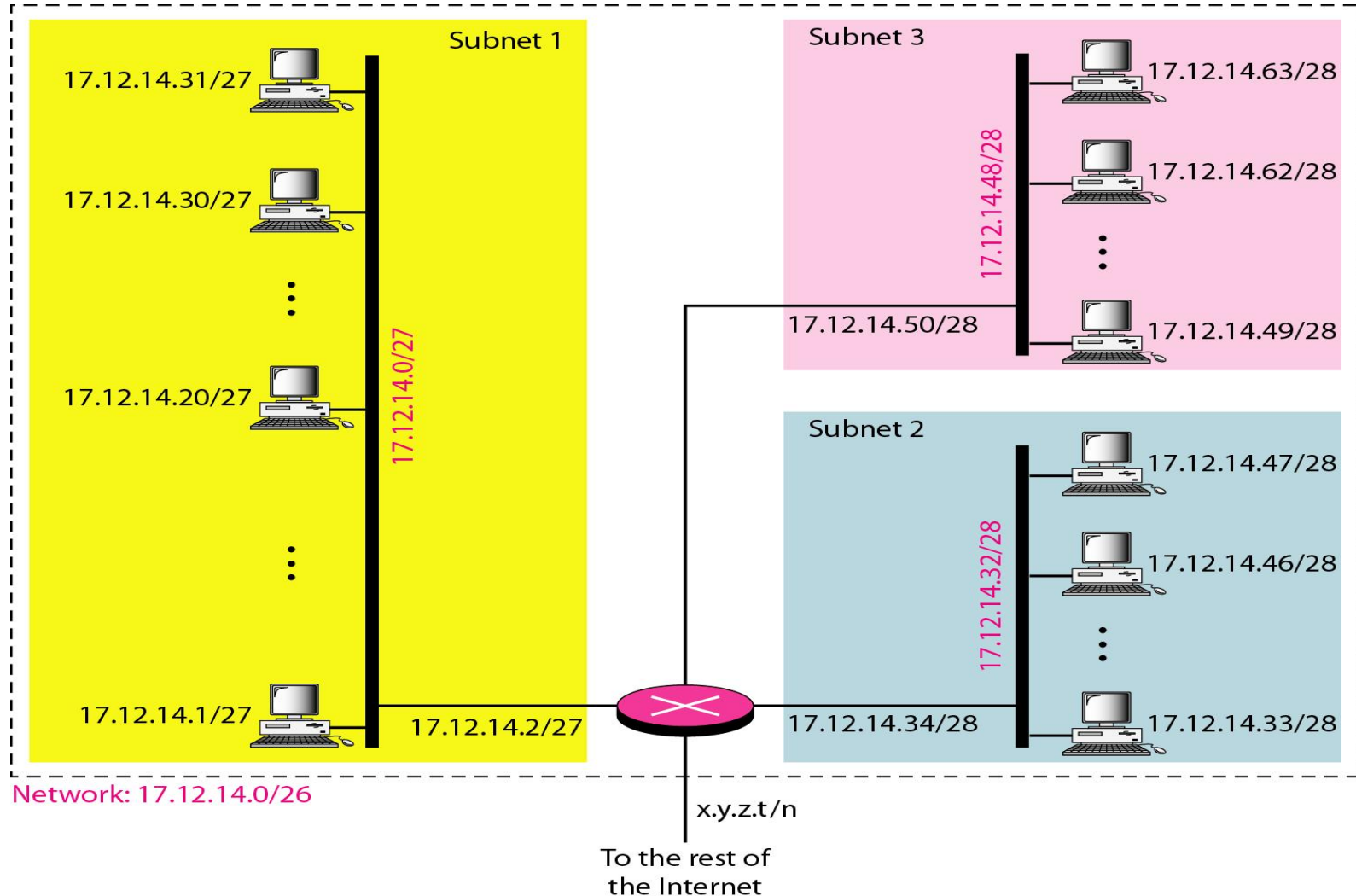
### ***Solution:***

***Subnet 1: 17.12.14.0/27 to 17.12.14.31/27***

***Subnet 2: 17.12.14.32/28 to 17.12.14.47/28***

***Subnet 3: 17.12.14.48/28 to 17.12.14.63/28***

# Example 11 (continued)





## *Example 12 - Address Allocation*

*An ISP is granted a block of addresses starting with 190.100.0.0/16 (65,536 addresses). The ISP needs to distribute these addresses to three groups of customers as follows:*

- a. The first group has 64 customers; each needs 256 addresses.*
- b. The second group has 128 customers; each needs 128 addresses.*
- c. The third group has 128 customers; each needs 64 addresses.*

*Design the subblocks and find out how many addresses are still available after these allocations.*



## *Example 12 (continued)*

### *Group 1*

*For this group, each customer needs 256 addresses. This means that 8 ( $\log_2 256$ ) bits are needed to define each host. The prefix length is then  $32 - 8 = 24$ . The addresses are*

|                                                    |                        |                          |
|----------------------------------------------------|------------------------|--------------------------|
| <i>1st Customer:</i>                               | <i>190.100.0.0/24</i>  | <i>190.100.0.255/24</i>  |
| <i>2nd Customer:</i>                               | <i>190.100.1.0/24</i>  | <i>190.100.1.255/24</i>  |
| <i>...</i>                                         |                        |                          |
| <i>64th Customer:</i>                              | <i>190.100.63.0/24</i> | <i>190.100.63.255/24</i> |
| <i>Total = <math>64 \times 256 = 16,384</math></i> |                        |                          |



## *Example 12 (continued)*

### *Group 2*

*For this group, each customer needs 128 addresses. This means that 7 ( $\log_2 128$ ) bits are needed to define each host. The prefix length is then  $32 - 7 = 25$ . The addresses are*

|                                                     |                           |                           |
|-----------------------------------------------------|---------------------------|---------------------------|
| <i>1st Customer:</i>                                | <i>190.100.64.0/25</i>    | <i>190.100.64.127/25</i>  |
| <i>2nd Customer:</i>                                | <i>190.100.64.128/25</i>  | <i>190.100.64.255/25</i>  |
| <i>...</i>                                          |                           |                           |
| <i>128th Customer:</i>                              | <i>190.100.127.128/25</i> | <i>190.100.127.255/25</i> |
| <i>Total = <math>128 \times 128 = 16,384</math></i> |                           |                           |

## *Example 12 (continued)*

### *Group 3*

*For this group, each customer needs 64 addresses. This means that 6 ( $\log_2 64$ ) bits are needed to each host. The prefix length is then  $32 - 6 = 26$ . The addresses are*

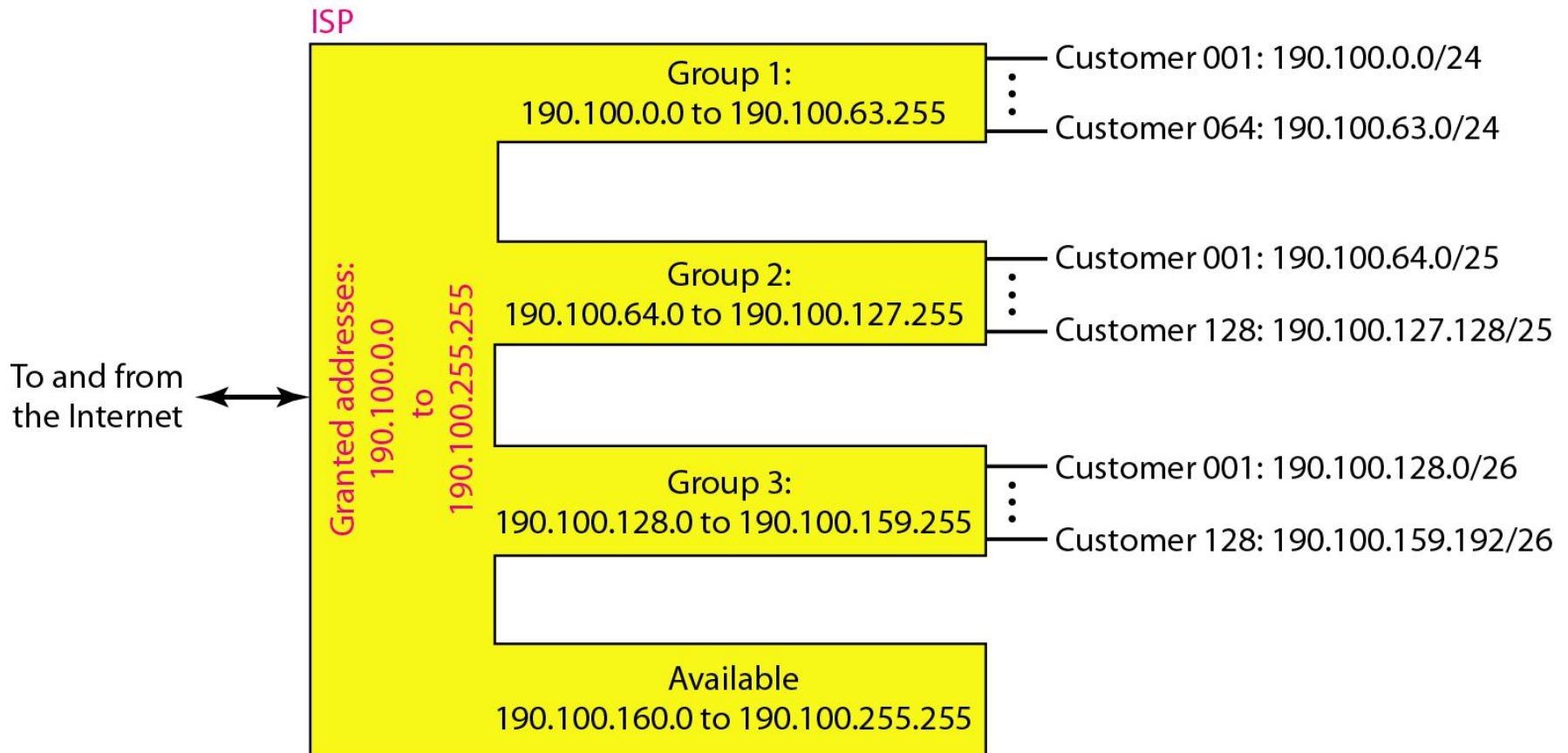
|                                                  |                           |                           |
|--------------------------------------------------|---------------------------|---------------------------|
| <i>1st Customer:</i>                             | <i>190.100.128.0/26</i>   | <i>190.100.128.63/26</i>  |
| <i>2nd Customer:</i>                             | <i>190.100.128.64/26</i>  | <i>190.100.128.127/26</i> |
| <i>...</i>                                       |                           |                           |
| <i>128th Customer:</i>                           | <i>190.100.159.192/26</i> | <i>190.100.159.255/26</i> |
| <i>Total = <math>128 \times 64 = 8192</math></i> |                           |                           |

*Number of granted addresses to the ISP: 65,536*

*Number of allocated addresses by the ISP: 40,960*

*Number of available addresses: 24,576*

## Example 12 (continued)



## ***Address Allocation – Problem 1***

***An organization is granted the block 211.17.180.0/24. The administrator wants to create 32 subnets.***

- a. Find the subnet mask.***
  - b. Find the number of addresses in each subnet.***
  - c. Find the first and last addresses in subnet 1.***
  - d. Find the first and last addresses in subnet 32.***
- Design the subblocks.***

## *Address Allocation - Problem 2*

*An organization is granted a block of addresses with the beginning address 14.24.74.0/24. The organization needs to have 3 subblocks of addresses to use in its three subnets: one subblock of 10 addresses, one subblock of 60 addresses, and one subblock of 120 addresses. Design the subblocks.*



## ***Address Allocation - Problem 3***

***An ISP is granted a block of addresses starting with 150.80.0.0/16. The ISP wants to distribute these blocks to 2600 customers as follows.***

***a. The first group has 200 medium-size businesses; each needs 16 addresses.***

***b. The second group has 400 small businesses; each needs 8 addresses.***

***c. The third group has 2000 households; each needs 4 addresses.***

***Design the subblocks and give the slash notation for each subblock. Find out how many addresses are still available after these allocations.***



## *Address Allocation - Problem 4*

*An ISP is granted a block of addresses starting with 150.80.0.0/16. The ISP wants to distribute these blocks to 2600 customers as follows.*

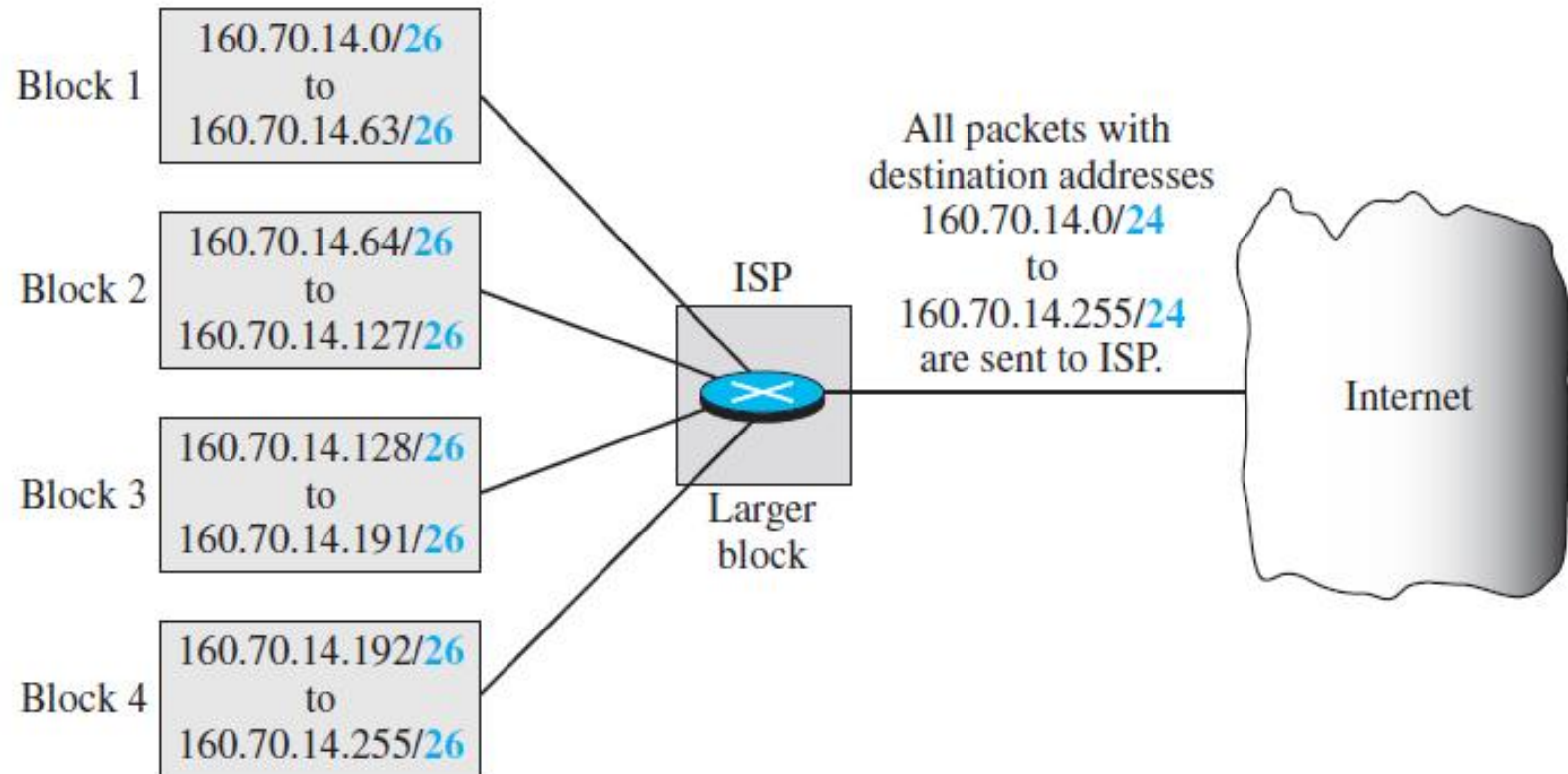
*a. The first group has 200 medium-size businesses; each needs 128 addresses.*

*b. The second group has 400 small businesses; each needs 16 addresses.*

*c. The third group has 2000 households; each needs 4 addresses.*

*Design the subblocks and give the slash notation for each subblock. Find out how many addresses are still available after these allocations.*

# ADDRESSES AGGREGATION



# ADDRESSES AGGREGATION

*1st Customer:*      *190.100.0.0/24*      *190.100.0.255/24*  
*2nd Customer:*      *190.100.1.0/24*      *190.100.1.255/24*  
    ...  
*64th Customer:*      *190.100.63.0/24*      *190.100.63.255/24*  
*Total = 64 × 256 = 16,384*

*1st Customer:*      *190.100.64.0/25*      *190.100.64.127/25*  
*2nd Customer:*      *190.100.64.128/25*      *190.100.64.255/25*  
    ...  
*128th Customer:*      *190.100.127.128/25*      *190.100.127.255/25*  
*Total = 128 × 128 = 16,384*

**Group1**

**Group2**

**Group3**

*1st Customer:*      *190.100.128.0/26*      *190.100.128.63/26*  
*2nd Customer:*      *190.100.128.64/26*      *190.100.128.127/26*  
    ...  
*128th Customer:*      *190.100.159.192/26*      *190.100.159.255/26*  
*Total = 128 × 64 = 8192*

**Group1: 190.100.0.0/18 to 190.100.63.255/18**

**Group2: 190.100.64.0/18 to 190.100.127.255/18**

**Group3: 190.100.128.0/19 to 190.100.159.255/19**

**Final: 190.100.0.0/16 to 190.100.255.255/16**

# SPECIAL ADDRESSES

## ✓ ***This-host Address***

✓ **0.0.0.0/32** (Only address in the block) - It is used when-ever a host needs to send an IP datagram but it does not know its own address to use as the source address.

## ✓ ***Limited-broadcast Address***

✓ **255.255.255.255/32** (Only address in the block) - It is used whenever a router or a host needs to send a datagram to all devices in a network. The routers in the network, however, block the packet having this address as the destination; the packet cannot travel outside the network.

# SPECIAL ADDRESSES

## ✓ *Loopback Address*

- ✓ The block **127.0.0.0/8** - A packet with one of the addresses in this block as the destination address never leaves the host; it will remain in the host.

## ✓ *Private Addresses*

- ✓ Four blocks are assigned as private addresses: **10.0.0.0/8**, **172.16.0.0/12**, **192.168.0.0/16**, and **169.254.0.0/16**.

## ✓ *Multicast Addresses*

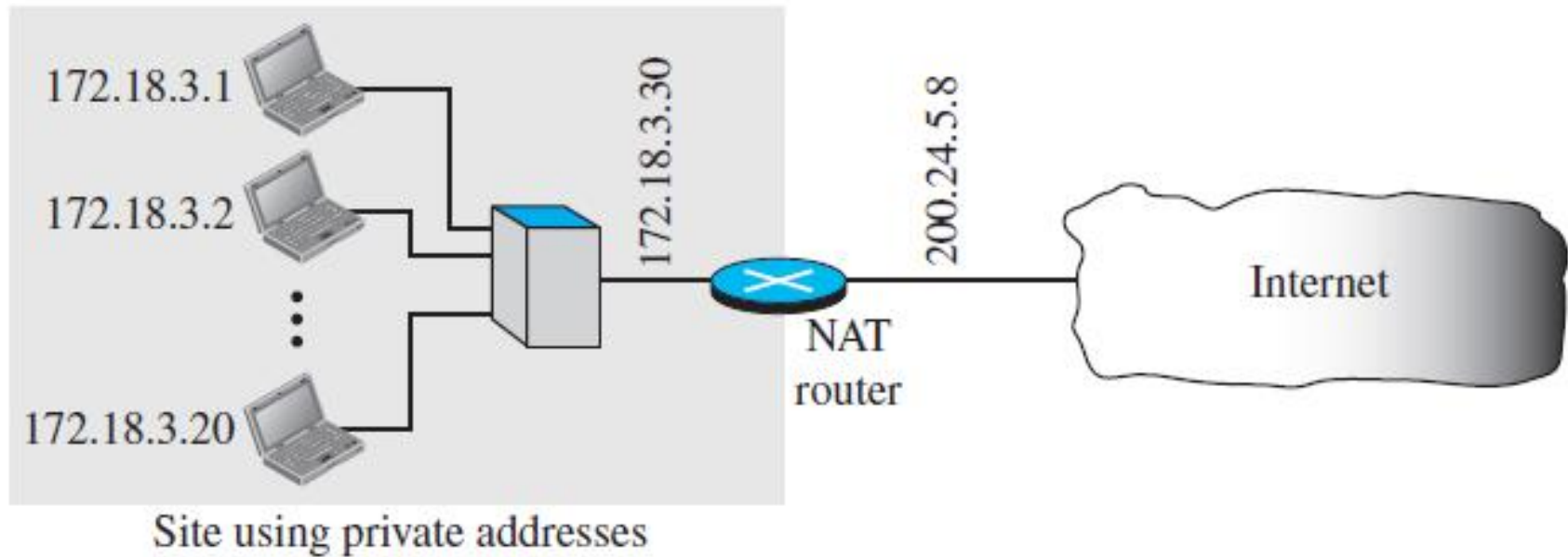
- ✓ The block **224.0.0.0/4** is reserved for multicast addresses

# *Network Address Translation*

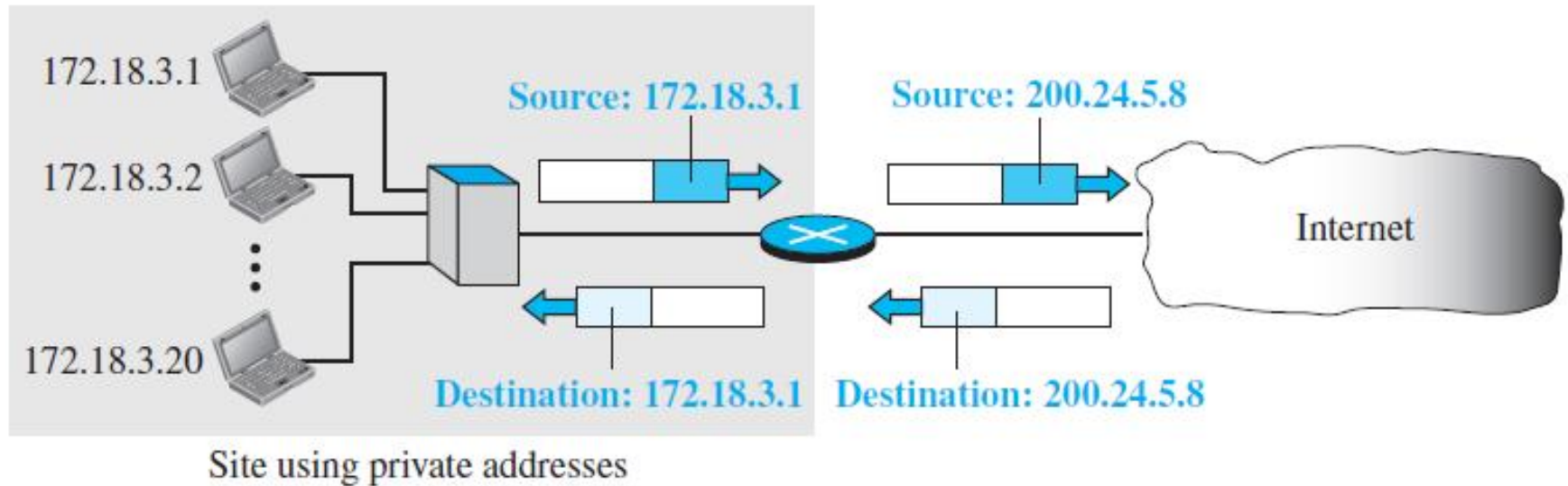
## *Addresses for private networks*

| <i>Range</i> |    |                 | <i>Total</i> |
|--------------|----|-----------------|--------------|
| 10.0.0.0     | to | 10.255.255.255  | $2^{24}$     |
| 172.16.0.0   | to | 172.31.255.255  | $2^{20}$     |
| 192.168.0.0  | to | 192.168.255.255 | $2^{16}$     |

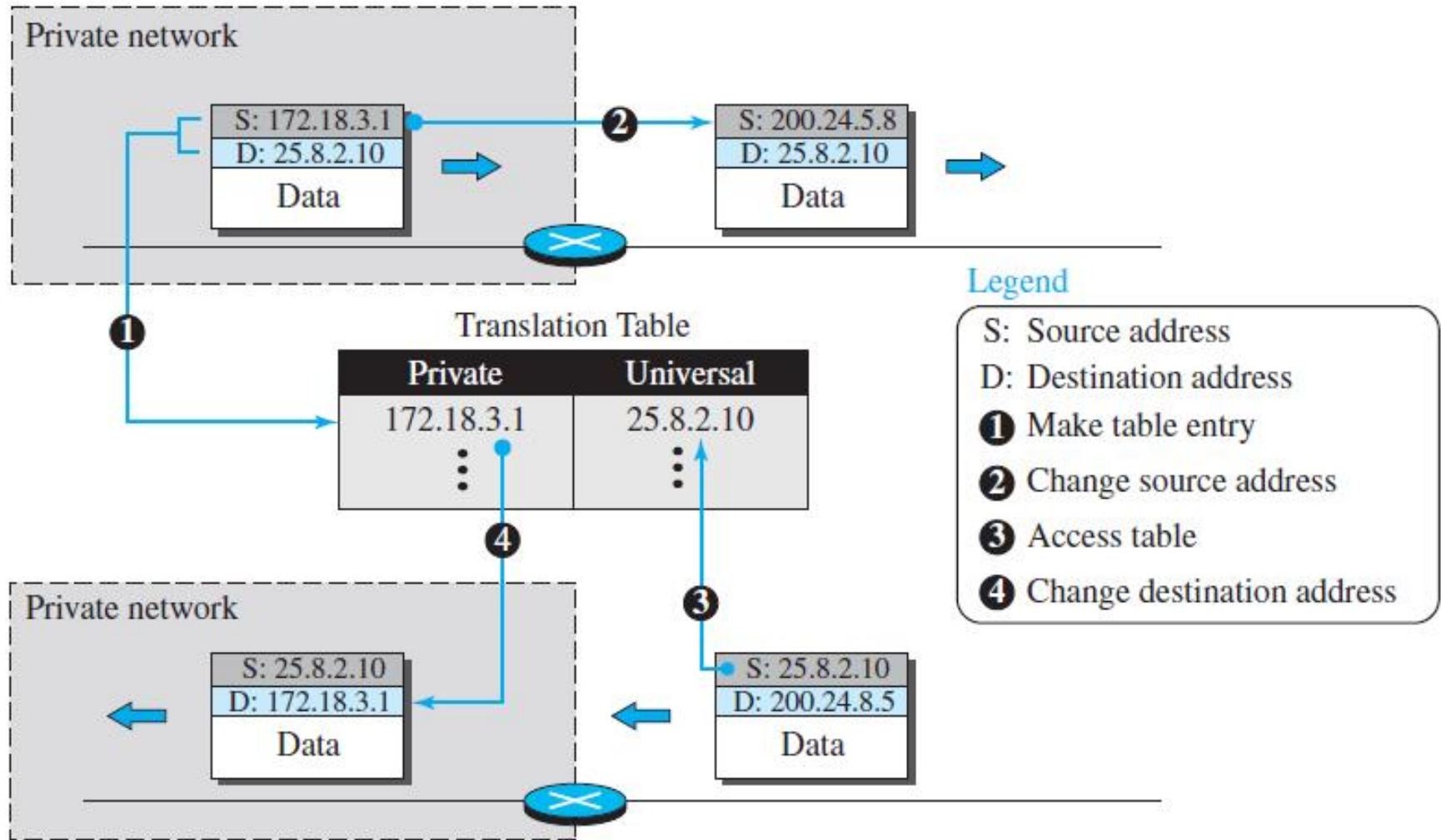
# *NAT implementation*



# *Addresses in a NAT*



# *NAT address translation Using One IP Address*



- ✓ *NAT address translation Using*
  - ✓ *One IP Address*
  - ✓ *Using a Pool of IP Addresses*
  - ✓ *Using Both IP Addresses and Port Addresses*

*Using Both IP Addresses and Port Addresses -  
Five-column translation table*

| <i>Private<br/>Address</i> | <i>Private<br/>Port</i> | <i>External<br/>Address</i> | <i>External<br/>Port</i> | <i>Transport<br/>Protocol</i> |
|----------------------------|-------------------------|-----------------------------|--------------------------|-------------------------------|
| 172.18.3.1                 | 1400                    | 25.8.3.2                    | 80                       | TCP                           |
| 172.18.3.2                 | 1401                    | 25.8.3.2                    | 80                       | TCP                           |
| ...                        | ...                     | ...                         | ...                      | ...                           |